CHAPTER FOUR: SYSTEM IMPLEMENTATION

Code Generation

The system was implemented as a web-based Inventory and Sales Management System using a combination of modern web technologies. The development followed the specifications defined in earlier chapters to ensure that all required functionalities were fully realized. The system uses PHP for server-side processing, JavaScript for client-side interaction, HTML for structuring web pages, and CSS for user interface design. The database was implemented using MySQL, accessed through the XAMPP server environment with phpMyAdmin used for database management. The code was developed using Sublime Text as the primary code editor.

Hardware Resources

The system was developed and tested using a standard personal computer equipped with sufficient processing power, memory (at least 4GB RAM), and storage. Input devices such as a keyboard and mouse were used for entering system data, while a display monitor provided the graphical interface for user interaction.

Software Resources

The software environment consisted of XAMPP, which provided Apache as the web server and MySQL as the database server. phpMyAdmin was used to create and manage database tables such as users, products, categories, suppliers, customers, and sales. A web browser was used to run and test the system.

The backend logic was implemented using PHP scripts such as main_server/server1, server2, usercrater, and loginserver. These scripts handle core operations including data insertion, updating, deletion, authentication, and retrieval. On the client side, JavaScript

functions were used to handle user interactions and asynchronous communication with the server using AJAX (XMLHttpRequest).

Actual Software Code Generation

The system functionalities were implemented according to the system requirements. Product management was implemented using JavaScript functions such as getnewproduct(), which collects product details including name, category, supplier, price, and quantity, and sends them to the server using a POST request. The PHP backend processes the request and stores the data in the MySQL database.

Customer and supplier management functionalities were implemented using functions such as getnewcustomer() and getnewsupplier(), which communicate with the server to store and update records. Category management is handled using getnewcategory().

User account creation and management were implemented using the createuser() function, which sends user details including name, email, password, and profile image to the server. Authentication is handled through the loginserver, where users log in using their email and password stored in the database. Only users approved by the administrator are allowed access, ensuring system security.

Sales functionality is implemented by allowing users to select products from the system. Once a product is selected, its details such as price are automatically retrieved, and the user inputs the quantity. The system then calculates the total sales value and profit in real time using JavaScript functions such as profiter(). The sales data is then sent to the backend and stored in the database.

The system also includes additional utility functions such as notifier() for displaying success or error messages, exportTableToExcel() for exporting data, and globaleditor() for retrieving existing records for editing. These functions improve usability and system efficiency.

Software System Integration

System integration involved combining the frontend interface, backend logic, and database into a unified working system. The frontend, developed using HTML and CSS, provides forms and dashboards through which users interact with the system. JavaScript enhances this interface by handling dynamic actions such as form submissions, calculations, and real-time updates.

AJAX (using XMLHttpRequest) is used to enable asynchronous communication between the client and server. This allows data to be sent and received without refreshing the page, improving system responsiveness. For example, when a user adds a product or records a sale, the data is sent to the server in the background, processed, and a response is returned instantly.

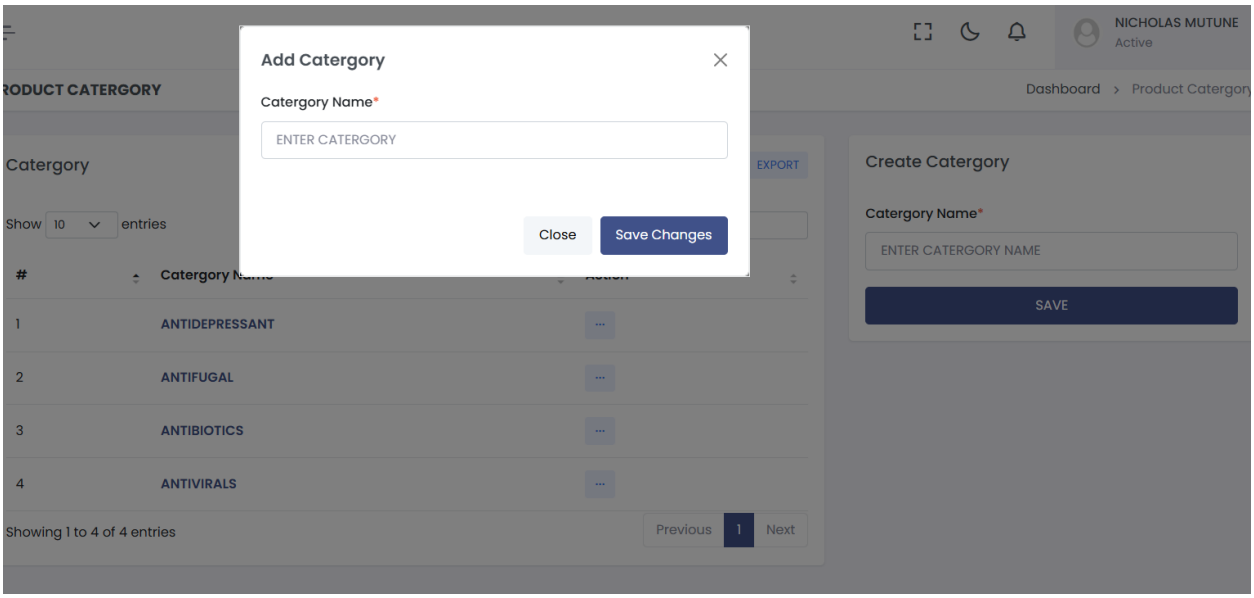
The PHP backend scripts process incoming requests and interact with the MySQL database. Data is stored in structured tables such as products, customers, suppliers, categories, users, and sales. The integration ensures that all components work together seamlessly, from data entry to storage and retrieval.

The system dashboard integrates various components by displaying key information such as daily sales, total sales trends, and top-selling products. Sales data is visualized using charts, including a line chart showing sales over seven days and a pie chart showing the most sold products based on quantity. These visualizations are generated dynamically based on data retrieved from the database.

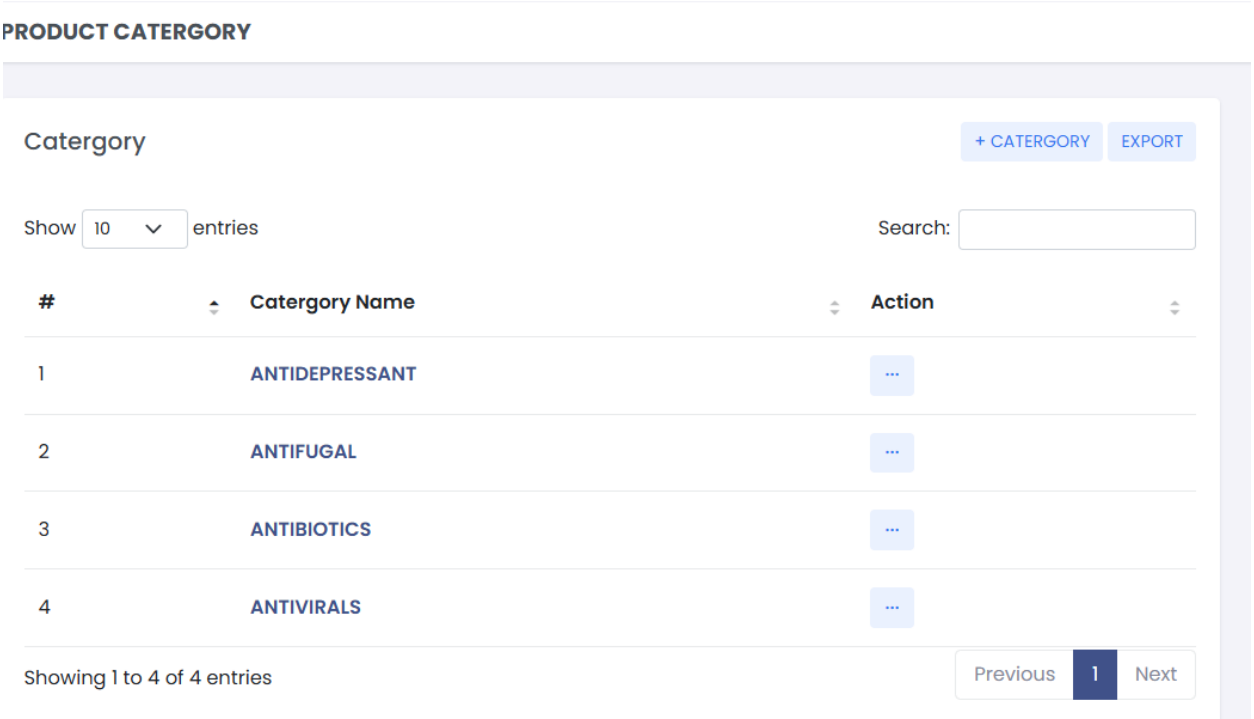
The overall system follows a client-server architecture, where the browser acts as the client and the PHP scripts on the XAMPP server act as the server. This architecture ensures efficient processing, modular design, and ease of maintenance.

Software System Testing

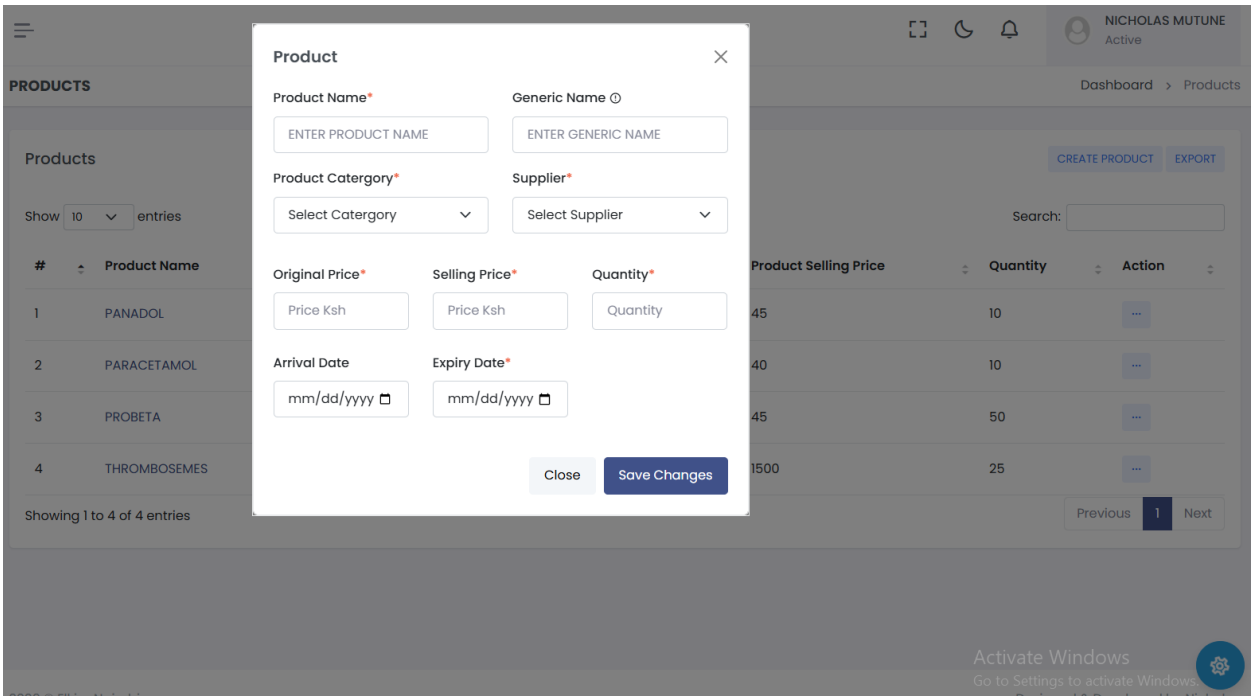
Software testing was conducted to verify that the system meets the specified requirements and functions correctly under different conditions. Testing involved executing system features, observing outputs, and comparing them with expected results. The following screenshots shows the working of the major functionalities of the system.



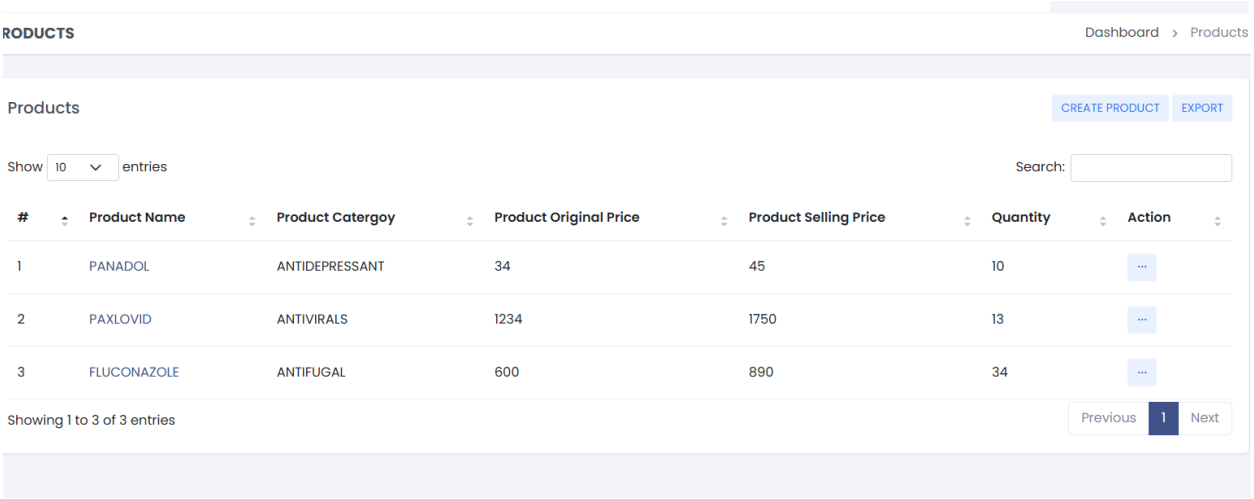
Results after adding product categories



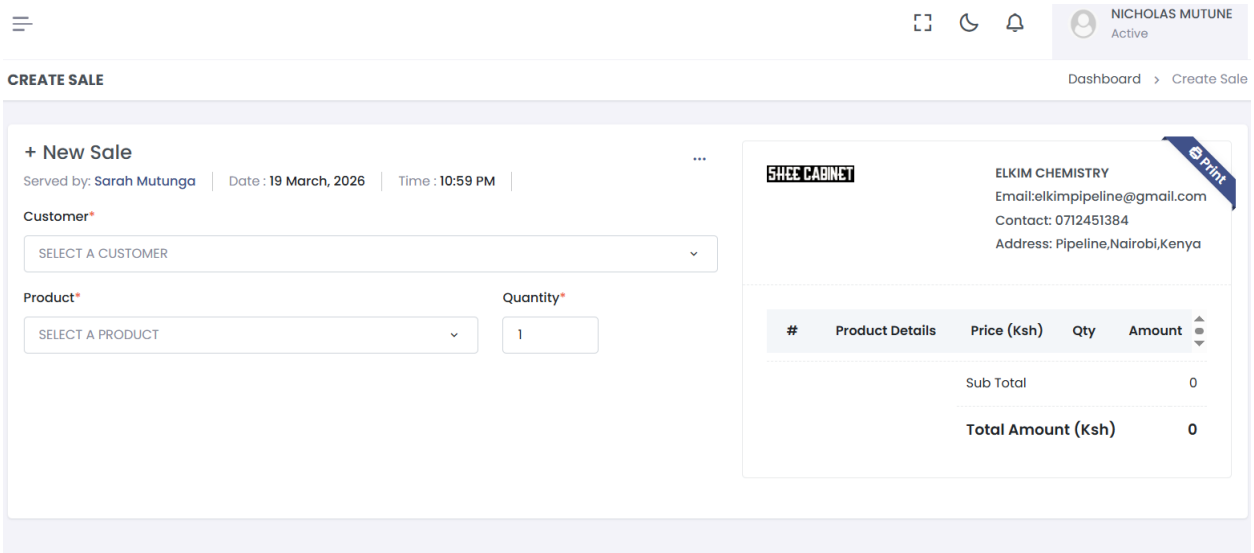
Adding products into the system is shown below



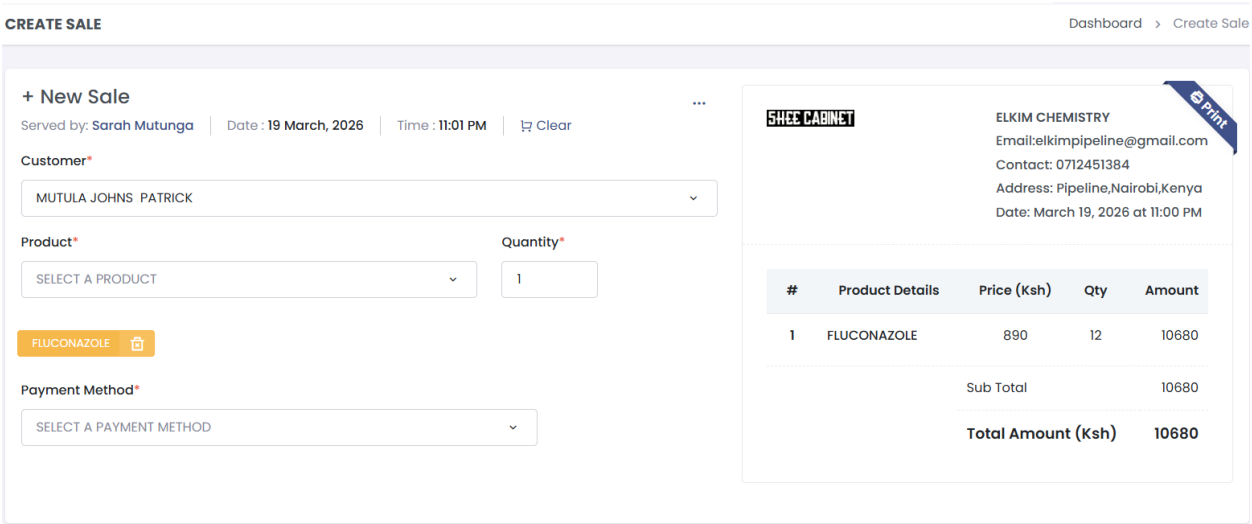
After adding products, they are displayed in the dashboard



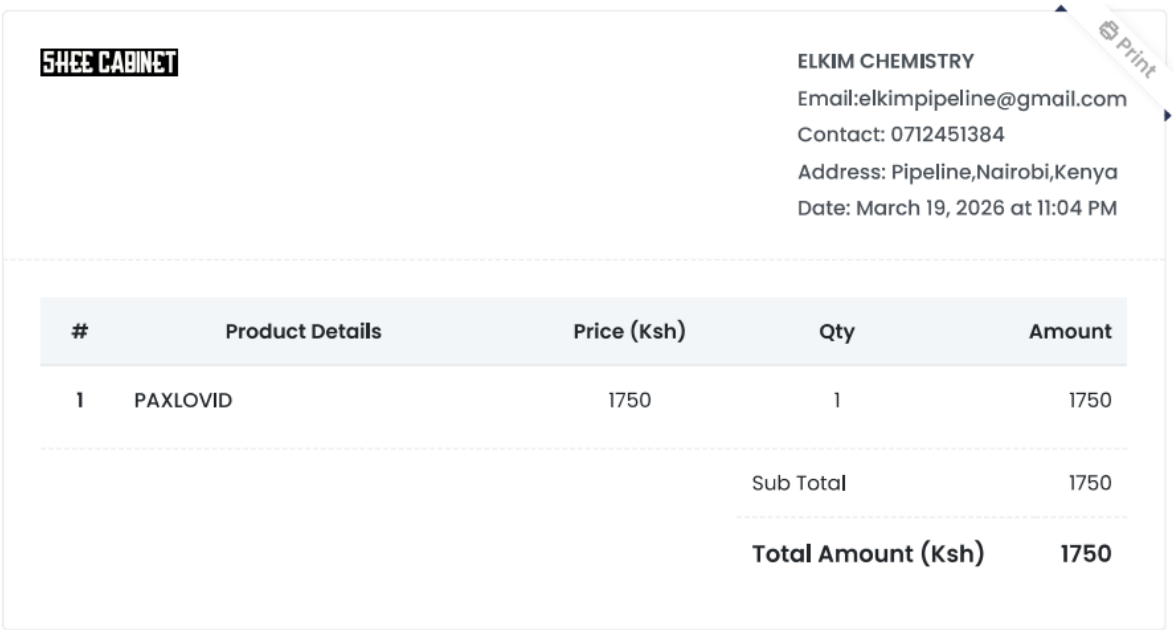
Below is a form that appears when a system user, employee wants to sell a product within the system



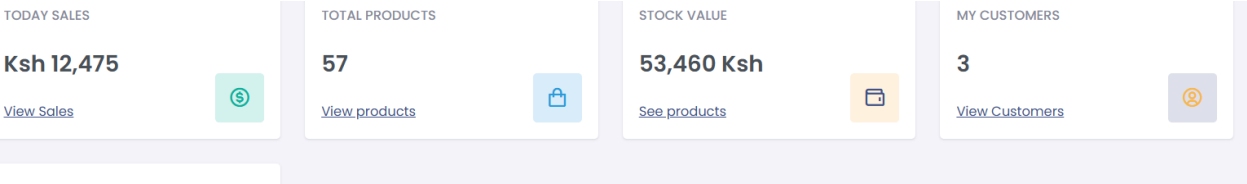
After inputting all the fields and saving, it appears like this, allowing the user to input the payment method used by a customer and save the sale, which is saved on the database and a report can be generated and stored within the permanent storage of a computer since it permits printing the receipt in PDF.



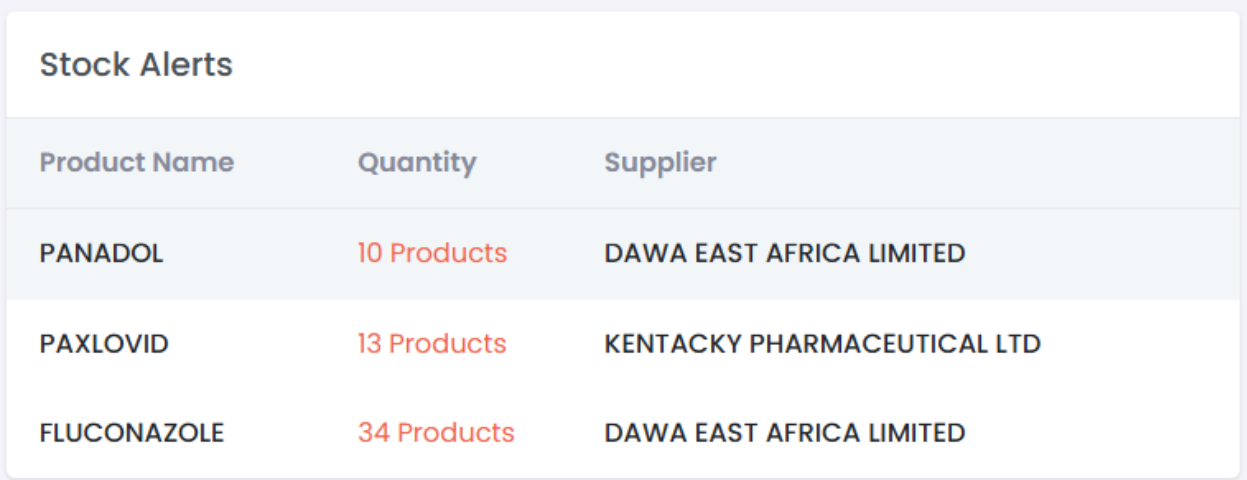
A receipt for each sale appears like this and can be downloaded and saved



Sales are recorded here



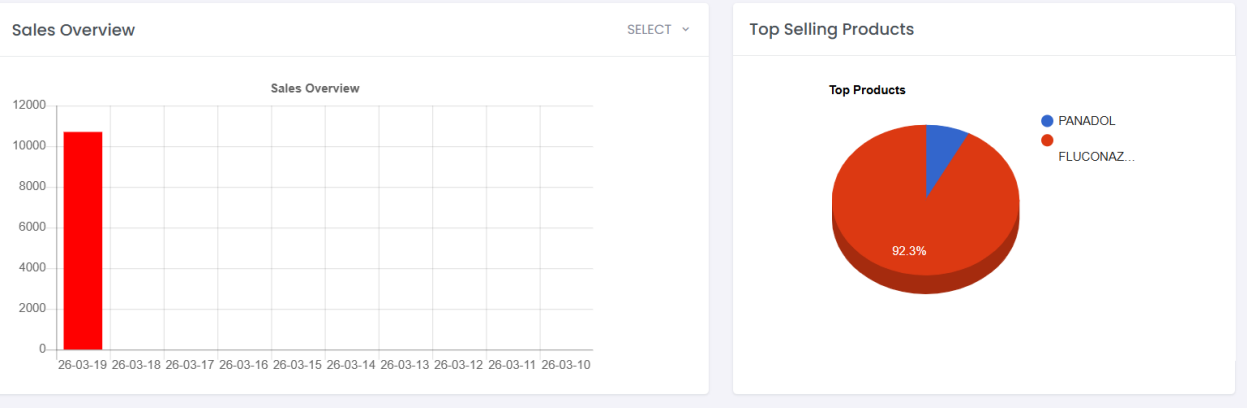
Stock alerts shows the remaining drugs as they are sold and reduces them as they are being sold and also adds and updates the correct number when products are added into the system



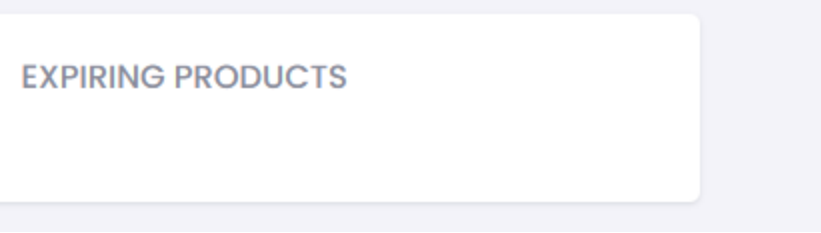
Stock Alerts

Product Name	Quantity	Supplier
PANADOL	10 Products	DAWA EAST AFRICA LIMITED
PAXLOVID	13 Products	KENTACKY PHARMACEUTICAL LTD
FLUCONAZOLE	34 Products	DAWA EAST AFRICA LIMITED

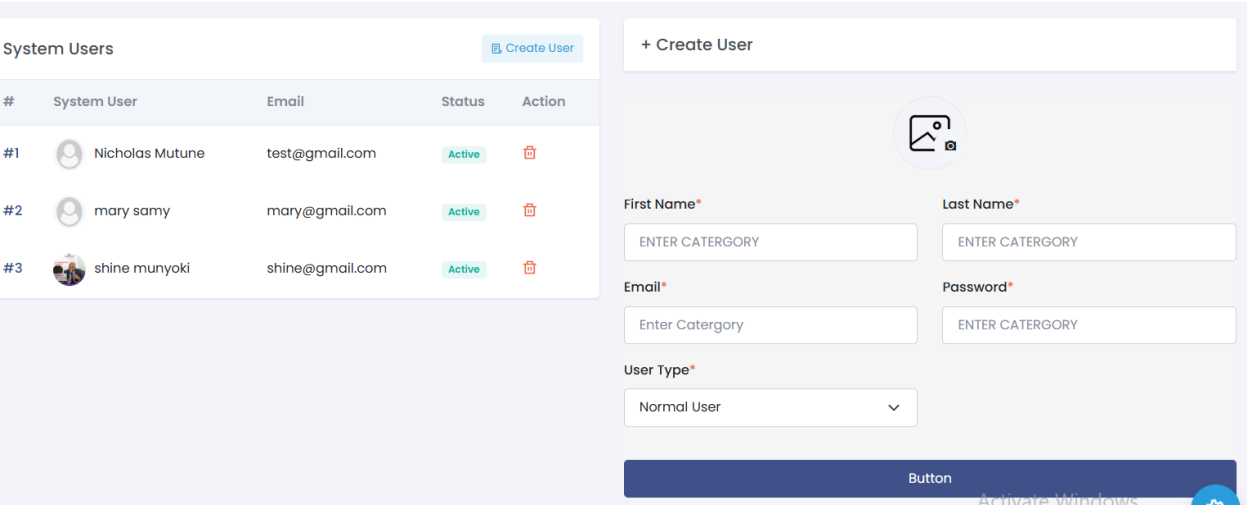
Sales overview shows total sales in a day in a chart that also shows total sales for each day, in the previous 7 days, and the pie chart shows the product that is sold in large numbers



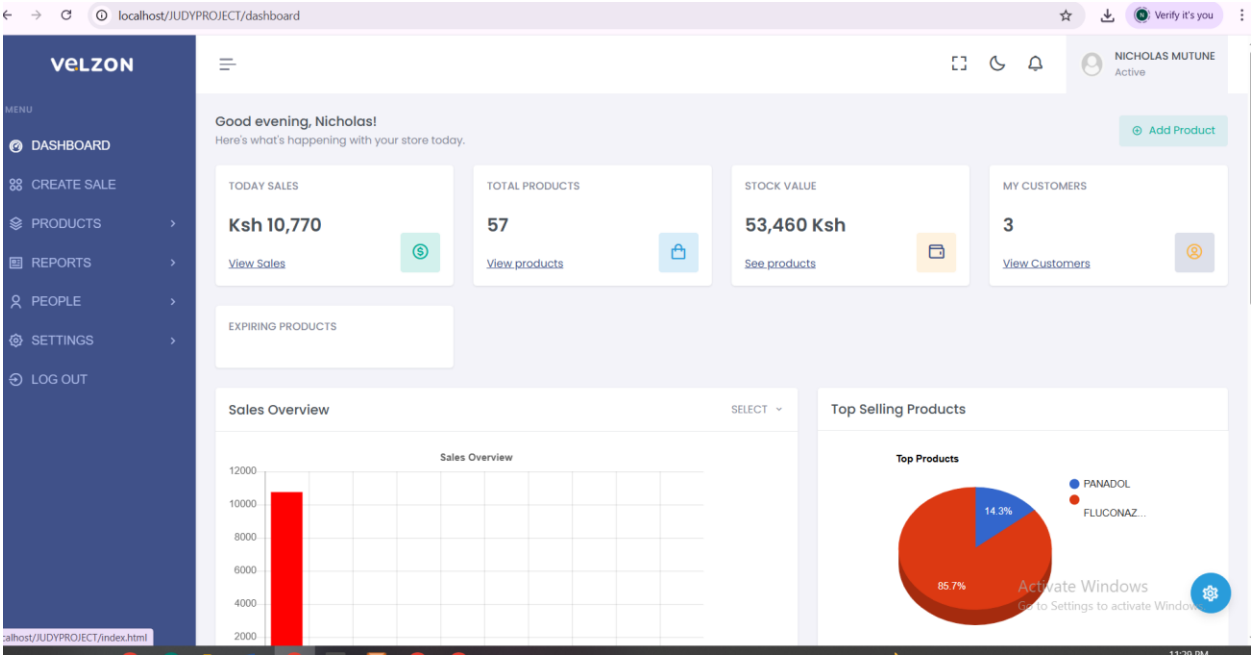
The products expiring in 5 months to come are displayed here, with a warning alert when logging in.

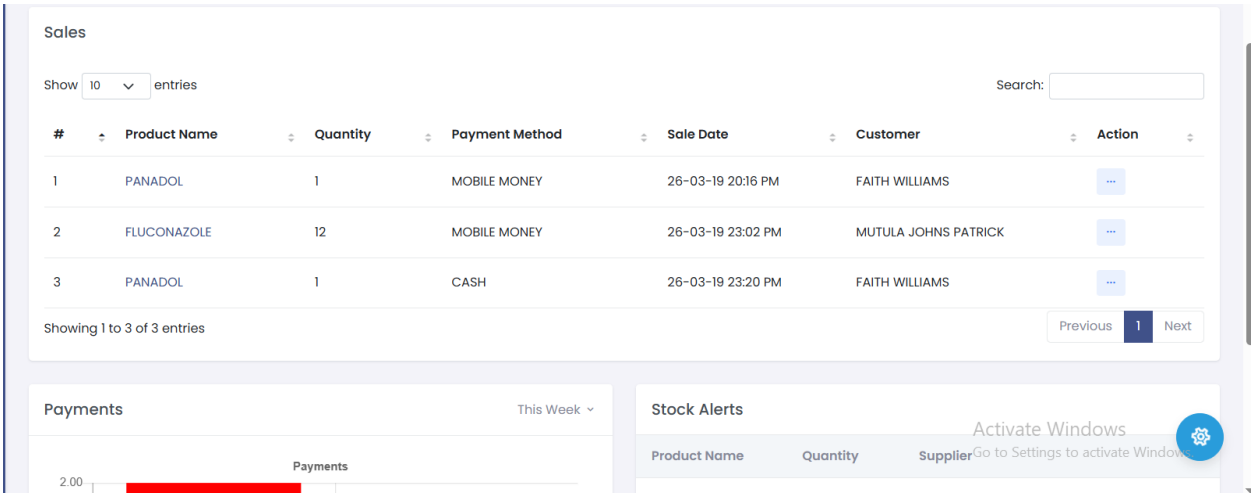


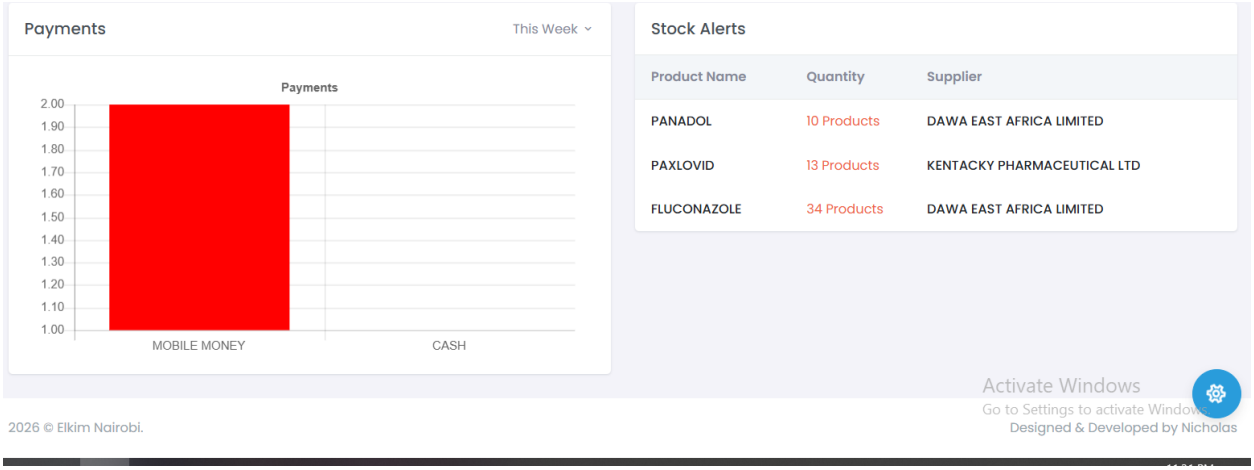
Eligible system users are managed here by the admin and one cannot change the login details unless authorized by the admin



Overall dashboard appearance below







Functional testing was performed on all major modules, including user authentication, product management, sales processing, customer management, supplier management, and category management. The login system was tested to ensure that only authorized users with valid email and password credentials can access the system. Invalid login attempts were correctly rejected, preventing unauthorized access.

Sales functionality was tested by selecting products, entering quantities, and verifying that the system correctly calculates total sales and profit. The system was also tested to confirm that sales are recorded accurately in the database and reflected correctly on the dashboard charts.

Validation testing was conducted both on the client side (JavaScript) and server side (PHP). Input fields were checked to ensure that required data is entered and that numeric fields

such as price and quantity contain valid values. This dual validation approach improves data accuracy and system reliability.

Integration testing was performed to ensure proper communication between system components. This included verifying that JavaScript correctly sends data to PHP scripts and that PHP correctly interacts with the MySQL database. The system responses were tested to ensure that appropriate notifications are displayed to the user.

Error testing was also carried out to identify and handle potential issues. For example, incorrect inputs, failed database operations, and missing data fields were tested. During testing, issues such as missing arrival and expiry dates in product records were identified, indicating the need to ensure proper data transmission and storage. These issues were addressed to improve system completeness.

The system was evaluated against the defined requirements, and the testing results confirmed that it performs its intended functions effectively, with accurate data processing, secure user authentication, and reliable system behavior.

Test Case ID	**Functionality**	**Input**	**Expected Output**	**Actual Result**	**Status**
TC01	User Login	Valid email & password	User is granted access to dashboard	Successful	Pass
TC02	User Login	Invalid email or password	Access denied with error message	Successful	Pass
TC03	Add Product	Complete product details	Product saved in database	Successful	Pass
TC04	Add Product	Missing required fields	System prevents submission or shows error	Successful	Pass
TC05	Record Sale	Select product + valid quantity	Total price and profit calculated correctly	Successful	Pass
TC06	Record Sale	Invalid quantity (e.g., text input)	Input rejected or error message displayed	Successful	Pass

TC07	Dashboard Charts	Existing sales data	Charts display correct sales trends and distribution	Successful	Pass
TC08	Add Customer	Valid customer details	Customer saved successfully	Successful	Pass
TC09	Add Supplier	Valid supplier details	Supplier saved successfully	Successful	Pass
TC10	Data Export	Export table to Excel	Excel file downloaded correctly	Successful	Pass
TC11	Product Dates Issue	Enter arrival & expiry dates	Dates saved in database	Initially Failed but later fixed Fixed	Pass